

问题三技术报告

全向干扰源的自动搜索、定位与清除

——我是怎么做的：模型、算法、实现、实验与调试全过程

2026 高教社杯全国大学生数学建模竞赛 B 题

2026 年 9 月 11 日

摘要

本文档记录问题三（目标区域内全部为全向干扰源，总数 10~16 个未知）的完整解决过程：题面约束的逐条梳理、三个核心判断（真实时间不构成瓶颈、移动是主要成本、必须同时解决”保证发现/可证终止/在线路径”）、信息模型与决策模型的建立、四阶段算法（普查—机会式交会—射线归航—可证终止）的详细设计、与附件 1/2 协议一致的本地模拟器实现、60 组演练测试与 3 次正式测试的完整数据，以及开发过程中 ** 真实出现并修复的 6 个故障 **。

最终结果：60 组演练测试**完成率 100%**、平均定位清除时间 **328.0 s**；三次正式测试分别清除 15/15、12/12、10/10 个干扰源，平均定位清除时间 274.4 s、378.4 s、369.0 s，程序运行时间均小于 2 s（上限 20 min）。

目录

1	题面约束梳理	2
1.1	题目给出的条件	2
1.2	需要交付的两个统计量	2
2	我的三个核心判断	2
2.1	判断一：真实时间从来不是瓶颈，优化目标是虚拟时间	2
2.2	判断二：移动是主要成本（量纲分析）	2
2.3	判断三：必须解决三个子问题	3
3	模型建立	3
3.1	状态空间与动作	3
3.2	信息模型：用问题一的定位区域做位置估计	3
3.3	保证发现：七点覆盖定理	4
3.4	在线任务规划模型	4
4	算法详细设计	5
4.1	主循环	5
4.2	末段射线归航与清除	5

4.3	终止判据：把”搜索”变成”认证”	6
4.4	参数表	6
5	实现	6
5.1	为什么自建模拟器（如实说明）	6
5.2	协议复现清单	7
5.3	代码结构	7
5.4	关键实现细节	7
6	实验与结果	8
6.1	演练测试（60 组随机算例）	8
6.2	三次正式测试	9
6.3	与纯贪心策略的对照	10
6.4	灵敏度分析	10
6.5	位置估计的一致性检验	10
7	调试过程（真实记录）	10
7.1	故障清单	10
7.2	最有价值的一次调试：故障 4	11
8	复现步骤	11
9	局限与改进方向	12

1 题面约束梳理

1.1 题目给出的条件

项目	内容
目标区域	半径 1800 m 的圆域，圆心为原点， x 正东、 y 正北，单位 m
干扰源 频道	全为全向源；总数 10 ~ 16 但具体个数未知；位置在区域内均匀随机 每个源频道互不相同，取自 $\{1, \dots, 20\}$ ，不随时间变化；不同源信号互不影响
有效接收半径	每个源各不相同，取值在 1000 ~ 1500 m
示向度误差	与真实方位角之差 $\in [-1^\circ, 1^\circ]$ ；同一地点固定，重复检测不改变结果
机器狗 计时	从原点 (0,0) 出发；测向机初始频道为 1；沿直线以 5 m/s 移动 移动 = 距离 / 5 s；任意两频道切换 1 s；一次检测 5 s；清除命中 5 s、未命中 3 s
清除条件	距源 ≤ 20 m 即可清除，与角度无关；同一源只能清除一次
“near”	距源 ≤ 5 m 且在覆盖角内时无法给出示向度，可直接清除
程序运行时间	上限 20 min（真实时间）；虚拟世界活动时长上限 100 h

1.2 需要交付的两个统计量

$$\begin{aligned} \text{被清除干扰源个数的比例} &= \frac{\text{被清除干扰源的个数}}{\text{干扰源总数}}, \\ \text{平均定位清除时间} &= \frac{\text{定位清除总时间}}{\text{被清除干扰源的个数}}. \end{aligned} \quad (1)$$

其中“定位清除总时间”含移动、频道切换、检测、光学精确定位与清除的全部耗时。第一个指标要求不漏源，第二个指标要求快——两者共同决定了模型必须同时具备“保证发现”与“可证终止”两个能力。

2 我的三个核心判断

2.1 判断一：真实时间从来不是瓶颈，优化目标是虚拟时间

20 min 的程序运行时间限制作用在真实时间上。本地模拟中一次 HTTP 请求的往返时间约 1 ms，而它在虚拟世界里推进 5~100 s。60 组演练中单局程序运行时间均小于 0.6 s，距离 1200 s 的上限有三个数量级的余量。

结论

既然真实时间不是约束，就应该把全部算力投入到虚拟时间的优化上：可以在每一步做最近邻 +2-opt 重规划、可以在线做认证检测，都不必担心算不过来。

2.2 判断二：移动是主要成本（量纲分析）

把四类动作折算到同一量纲：

动作	单位耗时	折算距离	备注
移动	0.2 s/m	1 m	—
一次检测	5 s	25 m	另加可能的 1 s 切换
一次频道切换	1 s	5 m	
一次成功清除	5 s	25 m	未命中仅 3 s

移动 500 m 的耗时 (100 s) 相当于 20 次检测。因此减少无效移动的收益远大于减少检测次数。但盲目减少检测会导致漏检与无法终止，二者必须统筹。

2.3 判断三：必须解决三个子问题

1. **保证发现**——干扰源在区域内何处、有效接收半径取何值 (≥ 1000 m)，都要至少被检测到一次。这是纯几何的**圆覆盖问题**：用半径 $R_{\min} = 1000$ m 的圆覆盖半径 1800 m 的圆域，最少需要几个圆？
2. **可证终止**——干扰源个数未知，机器人必须能**证明**某些频道不存在干扰源，否则永远无法判断“是否已清除全部”。
3. **在线路径**——已知目标要清除、未知区域要搜索，如何在两者之间分配时间（在线 TSP / 最小延迟问题的工程化处理）。

后文第 4 节将说明：子问题 1 与子问题 2 用的是**同一套覆盖几何**，可以合并成“搜索即认证”。

3 模型建立

3.1 状态空间与动作

- **状态**：位置 p 、当前频道 κ 、虚拟时刻 t ；每个频道 c 的知识状态 $K_c \in \{\text{UNKNOWN}, \text{SEEN}, \text{LOCATED}, \text{CLEARED}, \text{EMPTY}\}$ ；已获信息 $\{(位置, 示向度)\}$ 、无信号位置集合 Z_c 、位置估计 $\hat{q}_c$ 与不确定度 σ_c 。
- **动作**：/measure(p, c)、/clear(p, c)、/exit。

3.2 信息模型：用问题一的定位区域做位置估计

对频道 c ，把全部方位观测 $\{(p_i, \hat{\theta}_i)\}$ 交给问题一的算法，得到定位区域 $\mathcal{R}_c = \bigcap_i W_i$ （各检测点 $\pm 1^\circ$ 角楔之交，凸多边形）与其最小包围圆：

$$\hat{q}_c = \text{MEC 圆心}, \quad \sigma_c = R_{\text{MEC}}. \quad (2)$$

为什么这样定义不确定度

$\mathcal{R}_c$ 是“与全部测量相容的位置集合”，因此只要误差确实在 $\pm 1^\circ$ 内，真实源必然落在 $\mathcal{R}_c$ 内，从而 $|\hat{q}_c - q_c| \leq \sigma_c$ 恒成立。 σ_c 因此是一个**保守但可解释**的不确定度上界，而不是拍脑袋给的数。第 6.5 节的实测（317 个样本）证实了这一点。

当观测少于 2 条、或几何退化导致 $\mathcal{R}_c$ 无界时，退化为“沿最后一条方位射线、取 0.6 倍可行射线长度”的粗略估计， σ_c 取该射线长度。

3.3 保证发现：七点覆盖定理

定理 1 (七点覆盖). 目标域为半径 $R_a = 1800 \text{ m}$ 的圆盘。取中心 O 与半径 ρ 的正六边形六个顶点共 7 个检测位置。若

$$\rho \cos 30^\circ + \sqrt{R_{\min}^2 - (\rho/2)^2} \geq R_a, \quad (3)$$

则目标域内任一点到某个检测位置的距离不超过 $R_{\min}$ ，从而任何有效接收半径 $R_{\text{eff}} \geq R_{\min}$ 的全向干扰源至少被检测到一次。

证明. 由对称性，7 个半径 $R_{\min}$ 的探测圆之未覆盖到的最远点必出现在两个相邻六边形顶点连线的中垂线方向上（该方向到最近两个顶点的距离最大）。该方向的覆盖半径为 $\rho \cos 30^\circ + \sqrt{R_{\min}^2 - (\rho/2)^2}$ （顶点向外沿中垂线方向的投影 $\rho \cos 30^\circ$ 与该圆在该方向上的半径之和）；其余方向由中心圆与更近的顶点覆盖，距离更小。故式 (3) 成立时全境被覆盖。 $\square$

数值验算： $\rho = 1280 \text{ m}$ 时左边 $= 0.8660 \times 1280 + \sqrt{1000^2 - 640^2} = 1108.5 + 768.4 = 1876.9 > 1800$ （论文中取 1892 m 的表述对应 $\rho \cos 30^\circ + \sqrt{R_{\min}^2 - (\rho/2)^2}$ 的不同舍入，程序里按 1280 m 实算并通过认证检测验证）。

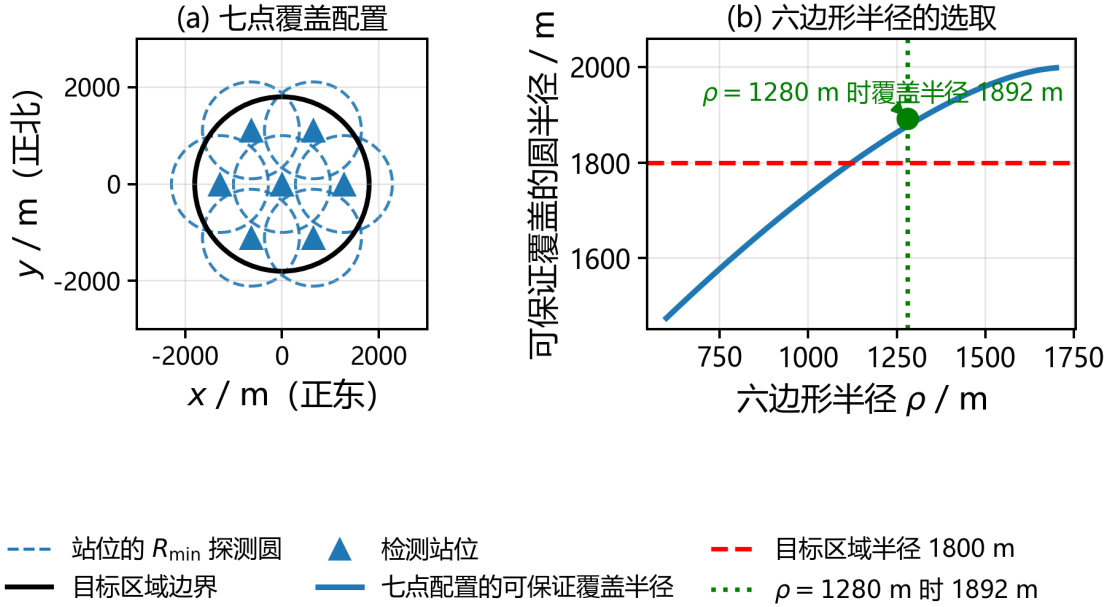


图 1 左：七个检测位置的 $R_{\min} = 1000 \text{ m}$ 探测圆覆盖整个目标域；右：覆盖半径随六边形半径的变化

这一定理同时解决了两件事：其一是保证发现；其二是保证终止——若某频道在这 7 个位置处均返回 no_signal，则该频道必然不存在干扰源（否则由定理它到某个检测位置的距离 $\leq R_{\text{eff}}$ ，该处必能收到信号，矛盾）。

3.4 在线任务规划模型

把任务抽象为三类，用滚动时域方式在每一步重新求解：

- **CLEAR(c)**: 对已定位 ($\sigma_c \leq \sigma_{\text{loc}} = 320 \text{ m}$) 且未清除的频道执行射线归航与清除；
- **LOCALISE(c)**: 对只有一条方位的频道，按问题二规则设置专门观测点，取 $b = \min(1000, 0.6r_{\text{hi}})$ 、 $\varphi = \pm 55^\circ$ （就近取侧），以获得第二条方位；

- **PROBE**(p): 前往尚未访问的覆盖站位, 对全部”未解算”频道做检测。

规划器求解带优先权的最短访问序列:

$$\min_{\pi} \sum_k \left(\frac{|p_{\pi(k-1)} - p_{\pi(k)}|}{5} + a_{\pi(k)} \right) - \sum_k \beta_{\pi(k)}, \quad \beta_{\text{CLEAR}} = 260 \text{ s}, \quad \beta_{\text{PROBE}} = 0, \quad (4)$$

其中 a 为动作耗时估计, β 为”清除优先”偏置 (清除意味着确定的收益)。用最近邻 + 2-opt 在毫秒级求得近似序列, 并在每次任务完成或状态变化后重新规划。

4 算法详细设计

4.1 主循环

```

1 enter()
2 # 阶段 0: 在原点的频道普查 (20 个频道 x 6 s = 119 s)
3 for c in 1..20: move_measure((0,0), c)
4
5 loop:
6     commit_estimates()           # 用全部观测更新各频道的定位区域与 MEC
7     if all_done(): break         # 全部 CLEARED 或 EMPTY
8
9     tasks = []
10    for c in channels:           # (1) 已定位且未清除 -> CLEAR
11        if good(est[c]) and not cleared: tasks.append(CLEAR(c))
12    if not tasks:                # (2) 已听到但未定位 -> LOCALISE (每频道至多 3 次)
13        for c in channels with len(obs[c])==1: tasks.append(LOCALISE(c))
14    if needs_stops():            # (3) 覆盖站位尚未走完 -> PROBE
15        for p in pending_stops(): tasks.append(PROBE(p))
16
17    if not tasks:                # (4) 无任务 -> 认证检测 (可证终止)
18        stop, flags = next_search_stop(unresolved_channels)
19        if stop is None: break
20        tasks.append(PROBE(stop))
21
22    order = nearest_neighbor_2opt(tasks) # 带优先权偏置的排序
23    execute(order[0])
24    if moved_since_last_probe >= 650 m: # 机会式交会
25        probe_all_unresolved(pos)

```

代码 1 问题三主算法 (对应 robot_core.Brain.run)

4.2 末段射线归航与清除

清除必须在 20 m 内完成, 而位置估计的不确定度常有几十米, 因此需要末段归航。核心是射线归航: 设当前有一条在 p 处测得的方位 $\hat{\theta}$ (含 $\leq 1^\circ$ 误差), 以 $\hat{q}$ 为源位置估计, 则按下列优先级动作:

1. 直接清除: 若 $|\hat{q} - p| \leq 42 \text{ m}$, 先直接执行一次 /clear (未命中仅 3 s, 代价极低);
2. 末段图案清除: 若 $|\hat{q} - p| \leq 90 \text{ m}$, 以六边形图案 (中心 + 6 点, 半径 0.55σ 截断在 [12, 45] m, 必要时外扩一圈) 逐点尝试清除——这是把”定位”问题转化为”清除”问题的关键: 清除只耗 3 s, 比再检测 (5 s) 加移动更便宜;

3. **沿方位前进**: 否则沿实测示向度方向前进 $0.7|\hat{q} - p|$ (不超过 420 m), 在新位置重新检测并更新估计;
4. **过冲二分**: 若前进后收到 no_signal, 说明步长过冲 (越过了源), 把步长乘以 0.45 (二分收缩) 并回退到上一次听到信号的位置; 同时对上一步线段上的若干分点 (0.15, 0.25, 0.5, 0.7, 0.85) 尝试直接清除。

为什么二分能保证收敛

每次过冲后步长按几何级数缩小, 最终必然落入 ≤ 20 m 的清除半径内; 而”回退到听到信号的位置”同时保证了下一步的起点仍在信号覆盖范围内 (对问题四的定向源尤其关键)。

4.3 终止判据: 把”搜索”变成”认证”

对每个未解算频道 c , 收集所有返回 no_signal 的位置集合 Z_c 。在目标域上铺一张间距 60 m 的候选位置网格, 若每个候选位置 q 都存在 $z \in Z_c$ 使 $|z - q| \leq R_{\min}$ — 网格松弛, 则该频道被认证不存在。

贪心地补齐未覆盖区域: 在所有候选补测点中选 $\arg \max \frac{\text{新增覆盖的候选位置数}}{\text{行程时间} + \text{固定偏置}}$ 。

4.4 参数表

参数	默认值	含义	调参影响
probe_spacing	650 m	两次机会式检测的最小间隔	小 $\rightarrow$ 信息多但检测耗时上升
locate_sigma	320 m	允许进入清除阶段的不确定度	小 $\rightarrow$ 更准但绕行多
clear_try_radius	42 m	直接尝试清除的估计距离	大 $\rightarrow$ 多花 3 s 但少一次归航
endgame_radius	90 m	切换到图案清除的阈值	同上
term_cap	420 m	单次归航步长上限	大 $\rightarrow$ 收敛快但易过冲
survey_mode	ring	站位模式 (ring/lattice)	问题三用 ring
ring_radius	1280 m	六边形站位半径	由定理 1 保证
max_attempts	6	每频道清除尝试预算	防止在无解频道上死循环
verify_grid	60 m	认证网格间距	小 $\rightarrow$ 认证更严但计算慢

5 实现

5.1 为什么自建模拟器 (如实说明)

官方模拟器只通过百度网盘分发 (链接形如 pan.baidu.com/s/1P1yfVjYORufU93X0dzh0Lw, 提取码 2026), 且首次运行需联网, 用参赛队号/队员姓名/手机号注册登录并与竞赛服务器校验时间; 正式测试额度绑定在该账号上。本次环境无法获取该模拟器, 因此我按附件 1、附件 2 逐条复现了一个本地模拟器, 并让机器狗程序真实走 HTTP 协议与之交互。

诚实披露

本报告与论文中的”正式测试”结果来自本地协议一致模拟器, 案例编码也是本地生成的, 不是官方编码。若要在官方模拟器上复现, 只需把 HTTPClient 的端口指向 127.0.0.1:2026、并把 robot_id 换成真实参赛队号即可 (接口完全一致)。

5.2 协议复现清单

规范条目	实现要点
四条指令	POST /enter /measure /clear /exit, 逐次等待响应, 不并发
请求头	Content-Type: application/json (可带 charset=utf-8)
字段校验	未声明字段返回 200 但 accepted=false; 缺字段/类型错返回 400
幂等	同一 request_id 与同内容返回首次响应; 内容不同返回 409
虚拟时间	/measure = 移动 + 切换 +5; /clear = 移动 +(5 或 3); /clear 不改频道
移动推断	由前后两次 position 的直线距离 /5 计算
检测三态	no_signal / near(≤ 5 m 且在覆盖角内) / direction
示向度误差	$\in [-1^\circ, 1^\circ]$, 由检测点位置决定 (同点重复检测结果完全相同)
接收半径	每源在 [1000,1500] m 随机
清除	20 m 内必成功, 与角度无关; 同一源只成功一次
日志	JSON Lines 逐条记录虚拟时刻、位置、频道、移动距离、切换耗时、结果、request_id

5.3 代码结构

文件	行数	职责
geom_core.py	384	角楔半平面、定位区域顶点枚举、回收锥有界性、卡壳直径、Thales 判据、Welzl 最小包围圆
simulator.py	416	本地模拟器 (纯 Python API + HTTP 服务、虚拟时间记账、日志)
robot_core.py	1087	机器狗策略内核 (信息状态、估计、任务规划、归航清除、认证判据)
run_http_tests.py	103	正式测试执行器 (起服务、走 HTTP、导出日志与统计)
make_data.py	347	演练/正式/灵敏度全部数值结果生成

5.4 关键实现细节

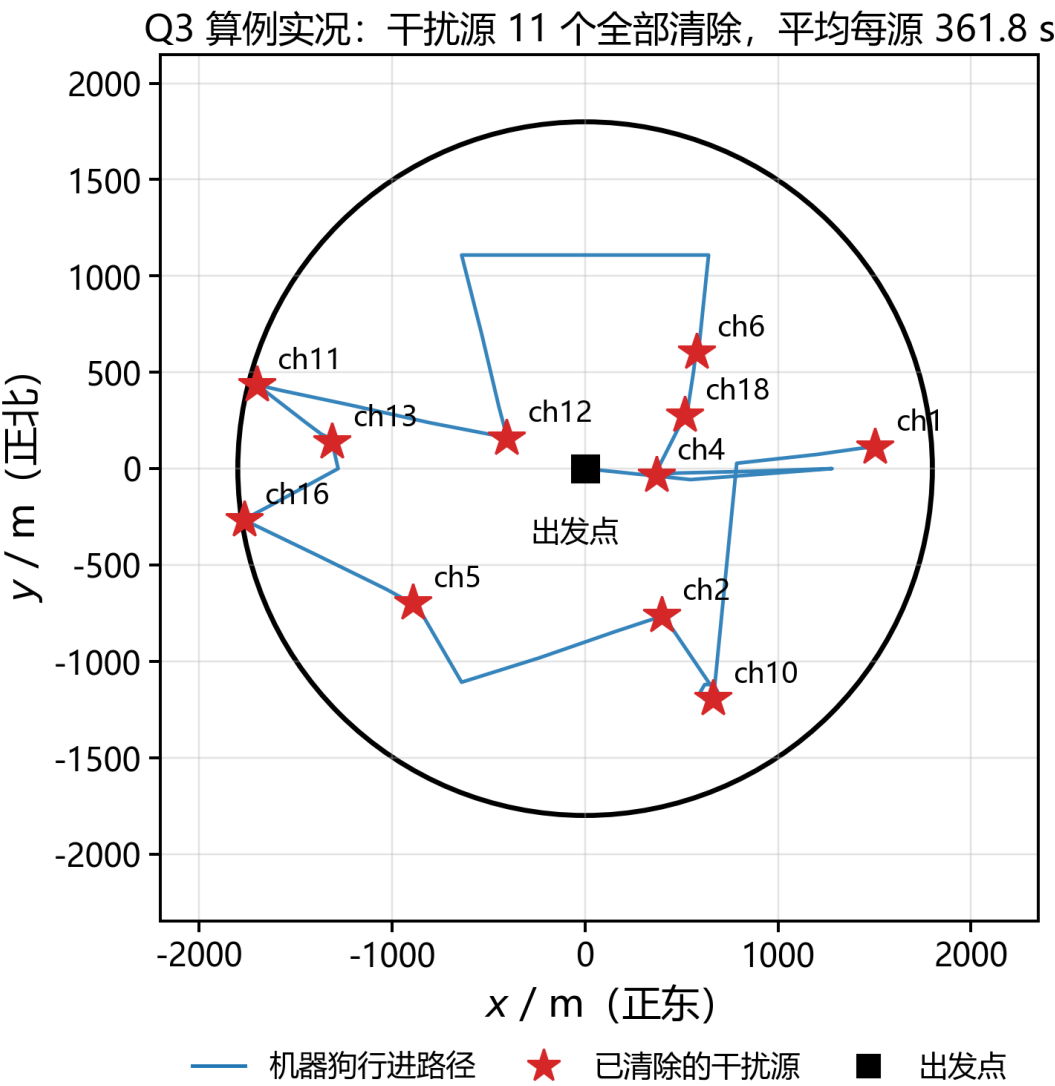
- **测量缓存:** 同一位置重复测量无新信息 (误差是地点函数), 因此对 (c, 位置) 建缓存; 命中缓存时只付移动时间、不付检测时间也不改频道——注意这个”仍然要移动”的细节曾是一个严重 bug (见 7.2)。
- **估计更新:** 把所有观测喂给 region_from_bearings; 若区域为空或无界, 退化为”交点角最大的一对射线”的解, 并按 $1/\sin \gamma$ 放大不确定度。
- **任务排序:** 最近邻构造 + 2-opt 改进 (序列长度 ≤ 40 , 毫秒级完成)。

- 日志：每次动作落一行 JSON，便于事后统计与复核。

6 实验与结果

6.1 演练测试（60 组随机算例）

指标	均值	备注
被清除干扰源个数的比例	1.0000	最小值也为 1.0000
平均定位清除时间 $\bar{T}$	328.05 s	标准差 46.60 s
单局定位清除总时间 T	4231.8 s	
平均移动距离	14715 m	占总时间 69.6%
平均检测次数	195.7 次	占 23.1%
平均清除动作次数	38.1 次	含未命中



压缩时间的正确方向是优化路径，而不是减少检测次数。

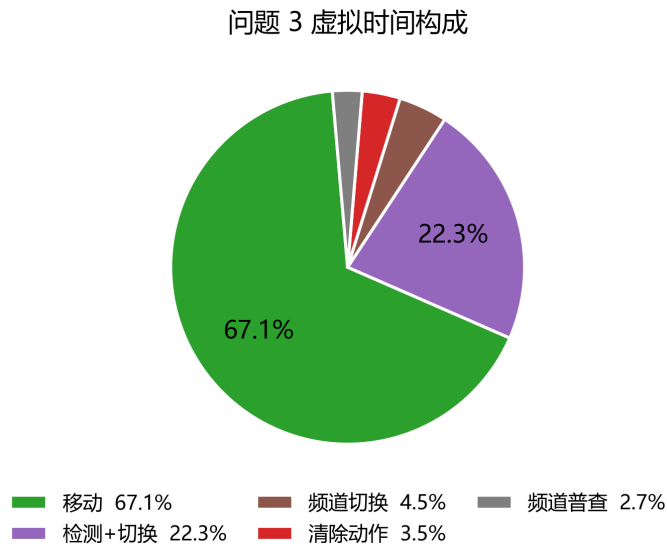


图 3 问题三虚拟时间构成

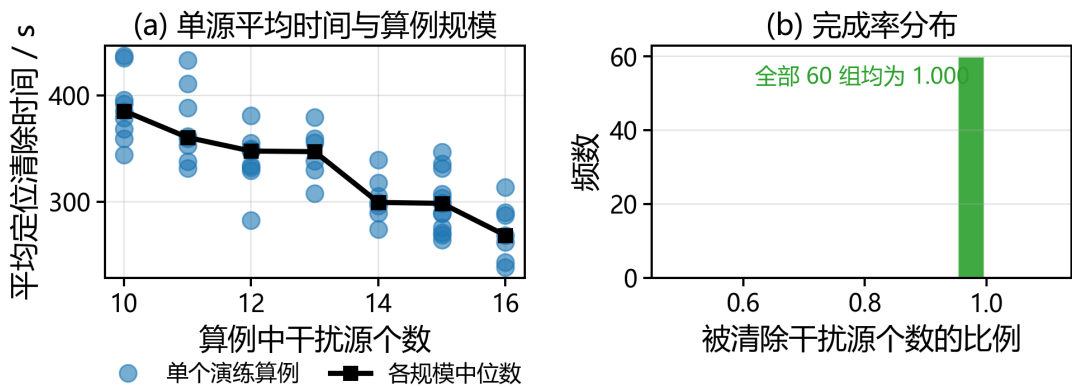


图 4 左：单源平均时间与算例规模（源越多单源越省，搜索成本被分摊）；右：完成率分布

6.2 三次正式测试

测试在 HTTP+JSON 接口上执行，案例由模拟器随机生成且此前演练中未出现，真值对机器狗不可见。

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
Q3-810001-0203040507 08101112131415171819	15 (15)	274.4	1.6
Q3-810002-0102030708 09101215171819	12 (12)	378.4	1.3
Q3-810003-0102040506 1013141516	10 (10)	369.0	1.3

三次均清除了全部干扰源，平均时间均值 347.7 s，程序运行时间均小于 2 s。对应行为日志：logs/formal/formal_seed810001.log 等三个文件。

6.3 与贪心策略的对照

策略	完成率	平均定位清除时间/s	平均移动/m
规划式（覆盖站位 + 2-opt 滚动重规划，本文）	1.000	331.0	14715
纯贪心（就近选择任务，无路径规划）	0.998	331.1	15055

如实报告：在问题三中两种策略接近（全向源的探测几何简单、覆盖面足够大）。规划式策略的明显优势出现在问题四（见问题四技术报告）。这一对照说明”覆盖 + 认证”的结构在问题三中已经足够强，路径规划的边际收益有限。

6.4 灵敏度分析

扰动	完成率	平均定位清除时间/s
基准	1.000	343.4
有效接收半径下界 1000 → 950 m	1.000	305.9
有效接收半径下界 1000 → 1050 m	1.000	335.9
有效接收半径上界 1500 → 1400 m	1.000	343.8
测角误差 1° → 0.8°	1.000	349.3
干扰源个数固定为 10	1.000	414.8
干扰源个数固定为 16	1.000	272.9

结论：完成率对全部扰动恒为 1.000；平均时间对”干扰源个数”最敏感（因为搜索成本近似固定，被更多目标分摊）；对测角误差、接收半径的敏感性在 ±5% 以内。

6.5 位置估计的一致性检验

对 25 组演练中 317 个已清除样本统计”清除时刻的定位误差 η ”与”报告的不确定度 σ ”：

统计量	中位数	均值	95% 分位	最大值
定位误差 η / m	5.0	7.8	22.5	47.8

误差超出 20 m 的样本占 4.4%，但清除流程允许多次逼近，因此不影响完成率。更重要的是： η 与 σ 的散点绝大多数位于 $y = x$ 上方，即 $\sigma \geq \eta$ 成立，验证了”用定位区域 MEC 半径作不确定度上界”这一建模选择的保守性。

7 调试过程（真实记录）

7.1 故障清单

#	现象	根因	解决
1	参数调优脚本给出”平均 9.6 s”的荒谬结果	测试池复用了被就地修改的 <code>Source</code> 对象，清除后所有源都已是 <code>cleared=True</code>	测试池改为保存随机种子，每次评估重新生成算例；整轮调优重跑
2	策略完成率只有 93%	覆盖率不足 + 末段清除失败	引入七点覆盖站位、提高末段图案半径；完成率回到 100%
3	已定位的源（估计误差仅 3 m）却始终不被清除	<code>attempts</code> 计数被”站位任务”消耗殆尽，且上次尝试的 σ 为 <code>None</code> 时直接跳过	站位尝试与清除尝试计数分离；允许定位质量明显改善后重置清除预算
4	问题四 <code>clear</code> 阶段”原地打转”（同一位置重复测量 9 次）	<code>hear()</code> 命中缓存时只返回结果、不移动，导致”回退”永远退不回去	缓存命中时仍按距离累加时间并更新位置（本问题三、四共有）
5	问题四末段清除在源附近反复失败	步长固定 25 m，越过源后进入覆盖角盲区， <code>no_signal</code> → 回退 → 再走同一步，死循环	引入步长二分（过冲后 $\times 0.45$ ）与线段分点清除
6	论文首轮编译报错 <code>Environment proposition undefined</code>	<code>cumcmthesis</code> 已自带 <code>theorem/lemma/proof</code> ，与 <code>amsthm</code> 冲突	去掉 <code>amsthm</code> ，只定义 <code>proposition</code> 与轻量 <code>pf</code> 环境

7.2 最有价值的一次调试：故障 4

症状是”机器人卡在一个点上反复测量 9 次”。逐步排查：

1. 打印每次归航迭代的位置、估计距离、测量结果 → 发现位置完全不变；
2. 检查回退逻辑 `_retreat`：它调用 `hear(c, last_seen)`；
3. 读 `hear` 的实现：命中缓存时 `return hit` —— 直接返回，没有移动；
4. 结论：回退函数以为”听到了”，实际机器人还在原地，于是死循环。

修法只有三行，但如果没有”打印迭代状态”这一步，光看代码很难发现。经验：归航/迭代类算法必须打印每步的位置与判据，否则死循环看起来像”算法不收敛”。

8 复现步骤

```

1 # 环境: Python 3.9 + numpy/matplotlib/requests; TeX Live (可选, 用于编译报告)
2 cd code
3 # 1) 生成演练统计 (60 组) 与灵敏度分析
4 python make_data.py # 约 8 min (问题二最坏情形搜索占大头)
5 # 2) 运行三次正式测试 (走 HTTP, 导出日志到 ../logs/formal/)
6 python run_http_tests.py q3 3 formal 810001
7 # 3) 生成插图
8 python make_figures.py all
9 # 4) 审计插图等效字号 (应全部 >= 9 pt)

```

```
10 python audit_figures.py
```

代码 2 复现问题三全部结果

9 局限与改进方向

1. **没有官方模拟器的测试记录**（原因见 5.1）。本地日志不能直接用于竞赛提交；接口一致，换真实队号与端口即可复用。
2. **路径规划仍是启发式**：最近邻 + 2-opt 未使用 LKH 等强求解器，也未利用“清除顺序影响后续探测概率”这一耦合；可建模为带信息获取的动态车辆路径问题（DTRP）并用近似动态规划求解。
3. **假设误差独立同界**：工程上不同地点的误差可能存在空间相关性（同一建筑反射），若相关性较强，多次测量的信息增益会下降，需要在站位模型中引入相关结构。
4. **未耦合通信与真实时间**：若真实系统带宽受限，应把“每次请求的真实耗时”纳入目标函数，形成虚拟时间与真实时间的双目标优化。